

# FastAPI for AI Services

Your RAG pipeline works in a notebook. Here's everything between that and an API real users can call.

 notebook.ipynb

**pipeline.run()**  
**works! ✓**

one user  
one request  
no failures



**production API**

**POST /query**

- validated • async
- streamed • rate-lim
- observed • secured

THOUSANDS OF USERS



AI Coach John

[Follow](#)



PROITBRIDGE  
DRIVER FOR BETTER FUTURE

 [Repost](#)

# Why FastAPI for AI Specifically

**i IT HAPPENS TO FIT THIS WORKLOAD ALMOST PERFECTLY**

## WHAT AI SERVICES NEED

## WHAT FASTAPI GIVES YOU

🕒 handle slow I/O calls

→ native `async/await`

≡ stream tokens out

→ `StreamingResponse`

🛡 validate messy input

→ `Pydantic` models

📦 load a model once

→ lifespan events

📄 document the API

→ auto OpenAPI docs

↔ typed request/response

→ Python type hints

## 💡 THE ASYNC POINT IS THE BIG ONE

An LLM call is mostly waiting on a network. A sync framework ties up a worker per request; FastAPI's async model lets one process hold hundreds of in-flight requests, which matters enormously when each one takes 2 to 4 seconds.



AI Coach John

Follow



PROITBRIDGE  
DRIVEN BY BETTER TOOLS

↻ Repost

# The Minimum Viable AI API

## </> START HERE, THEN HARDEN

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI(title="RAG Service", version="1.0.0")

class QueryRequest(BaseModel):
    question: str

class QueryResponse(BaseModel):
    answer: str

@app.get("/health")
async def health():
    return {"status": "ok"}

@app.post("/query", response_model=QueryResponse)
async def query(req: QueryRequest):
    answer = await pipeline.arun(req.question)
    return QueryResponse(answer=answer)
```

```
uv add fastapi uvicorn
uv run uvicorn main:app --reload --port 8000
```

## 📁 WHAT YOU GET FOR FREE

POST /query {"question": "... "}

- ✓ request validated against QueryRequest
- ✓ 422 returned automatically if it doesn't match
- ✓ response validated against QueryResponse

📄 [/docs page generated from the models](#)



# Pydantic Models: Your Input Contract

## ✓ VALIDATE AT THE EDGE SO YOUR PIPELINE NEVER SEES GARBAGE

```
class QueryRequest(BaseModel):
    question: str = Field(..., min_length=3, max_length=2000)
    top_k: int = Field(default=5, ge=1, le=20)
    temperature: float = Field(default=0.2, ge=0.0, le=1.0)
    mode: Literal["fast", "accurate"] = "fast"
    session_id: str | None = None

    @field_validator("question")
    @classmethod
    def no_empty_whitespace(cls, v: str) → str:
        if not v.strip():
            raise ValueError("question cannot be blank")
        return v.strip()

class QueryResponse(BaseModel):
    answer: str
    citations: list[Citation]
    tokens_used: int
    cost_usd: float
    latency_ms: int
    model: str
```

## ⊗ VALIDATION AT THE EDGE

bad request → 422 rejected  
never reaches your LLM • costs \$0

```
{
  "detail": [
    {
      "loc": ["body", "question"],
      "msg": "question cannot be blank"
    },
    {
      "loc": ["body", "top_k"],
      "msg": "less than or equal to 20"
    }
  ]
}
```

## ≡ RICH RESPONSE MODEL

Returning tokens, cost, and latency per request means your clients and dashboards get **observability for free**, without a separate metrics call.



# Loading the Model Once: Lifespan Events

## ! THE MISTAKE THAT MAKES EVERY REQUEST 30 SECONDS SLOW

```
ml_models = {}

@asynccontextmanager
async def lifespan(app: FastAPI):
    # STARTUP: runs once, before serving any request
    ml_models["embedder"] = SentenceTransformer("BAAI/bge-small-en-v1.5")
    ml_models["index"] = faiss.read_index("data/index.faiss")

    yield # ← the app serves requests here

    # SHUTDOWN: clean up
    ml_models.clear()

app = FastAPI(lifespan=lifespan)

@app.post("/query")
async def query(req: QueryRequest):
    vec = ml_models["embedder"].encode(req.question) # already in memory
```

### ✗ LOADING PER REQUEST

req 1  32s

req 2  32s

req 3  32s

↑ reloads model each time

### ✓ LOADING AT STARTUP

boot  30s

req 1  1.8s

req 2  1.8s

req 3  1.8s

↑ reused



# Dependency Injection

## CLEANER THAN A GLOBAL DICT, AND TESTABLE

```
from fastapi import Depends
from typing import Annotated

def get_pipeline() → RAGPipeline:
    return ml_models["pipeline"]

def get_current_user(api_key: str = Header(...)) → User:
    user = lookup_user(api_key)
    if not user:
        raise HTTPException(401, "invalid API key")
    return user

PipelineDep = Annotated[RAGPipeline, Depends(get_pipeline)]
UserDep = Annotated[User, Depends(get_current_user)]

@app.post("/query")
async def query(req: QueryRequest, pipeline: PipelineDep, user: UserDep):
    return await pipeline.arun(req.question, user_id=user.id)
```

## WHY DI BEATS GLOBALS

### TESTING

```
app.dependency_overrides[get_pipeline] = lambda: FakePipeline()
```

Swap in a fake with one line. No mocking of module-level globals required.

### REUSE

Auth · Rate limits · DB sessions

Declared once, attached to any endpoint.



# Streaming Responses

## 🕒 FIRST TOKEN IN 400MS INSTEAD OF A 3-SECOND BLANK SCREEN

```
async def generate_stream(question: str):
    docs = await retrieve(question)

    # send citations immediately, before generation starts
    yield f"data: {json.dumps({'type': 'citations', 'docs': docs})}\n\n"

    async for chunk in llm.astream(build_prompt(question, docs)):
        yield f"data: {json.dumps({'type': 'token', 'text': chunk})}\n\n"

@app.post("/query/stream")
async def query_stream(req: QueryRequest):
    return StreamingResponse(
        generate_stream(req.question),
        media_type="text/event-stream",
        headers={"Cache-Control": "no-cache", "X-Accel-Buffering": "no"},
    )
```

### ⌚ NON-STREAMING

2,800ms of blank  
User sees a spinner

### ⚡ STREAMING (SSE)

Citations sent in 120ms  
First token at 400ms  
User is already reading

**X-Accel-Buffering: no** stops nginx from buffering your stream into one chunk (a very common "streaming doesn't work in production" cause).



# Background Tasks & Long Jobs

## NOT EVERYTHING SHOULD BLOCK THE RESPONSE

```
from fastapi import BackgroundTasks

# Fire-and-forget: respond now, do extra work after
@app.post("/query")
async def query(req: QueryRequest, background: BackgroundTasks):
    answer = await pipeline.arun(req.question)
    background.add_task(log_to_analytics, req.question, answer)
    return {"answer": answer} # returns immediately

# Long jobs: return a job ID, poll for the result
@app.post("/index", status_code=202)
async def start_indexing(req: IndexRequest, background: BackgroundTasks):
    job_id = str(uuid.uuid4())
    background.add_task(run_indexing, job_id, req.documents)
    return {"job_id": job_id, "status_url": f"/jobs/{job_id}"}
```

## CHOOSING THE RIGHT PATTERN

| SCENARIO                   | PATTERN                   |
|----------------------------|---------------------------|
| < 100ms extra work         | Just await it inline      |
| Fire-and-forget (< 5s)     | BackgroundTasks           |
| Minutes-long job           | 202 + job ID + polling    |
| Survives restarts, retries | Real queue (Celery/Redis) |

⚠ BackgroundTasks live in-process: a restart loses them





# Error Handling

## 🛡️ NEVER LEAK A STACK TRACE; ALWAYS RETURN SOMETHING USEFUL

```
@app.exception_handler(RetrievalError)
async def handle_retrieval_error(request: Request, exc: RetrievalError):
    log.error("retrieval_failed", error=str(exc))
    return JSONResponse(
        status_code=503,
        content={"error": "search_unavailable", "retry_after": 5},
    )

@app.exception_handler(Exception) # catch-all, LAST resort
async def handle_unexpected(request: Request, exc: Exception):
    log.exception("unhandled_error", path=request.url.path)
    return JSONResponse(status_code=500,
        content={"error": "internal_error",
            "request_id": request.state.request_id})
```

## ☰ STATUS CODES FOR AI SERVICES

- 400** malformed request
- 401** missing / invalid API key
- 403** valid key, not allowed
- 422** validation failed (FastAPI does this)
- 429** YOUR rate limit hit (or upstream)
- 500** unexpected error on your side
- 503** dependency is down (vector DB, LLM)
- 504** upstream timed out

**⚠️ Never return 200 with an error message in the body. Clients can't distinguish success from failure.**



# Rate Limiting & Quotas

## TWO DIFFERENT PROTECTIONS, TWO DIFFERENT MESSAGES

```
# Rate limit: protects your infrastructure
@app.post("/query")
@limiter.limit("20/minute")
async def query(request: Request, req: QueryRequest, user: UserDep):

    # Quota: protects your margins
    if user.tokens_this_month >= user.plan.monthly_token_limit:
        raise HTTPException(
            status_code=402,
            detail={"error": "quota_exceeded", "upgrade_url": "/pricing"},
        )

    result = await pipeline.arun(req.question)
    await record_usage(user.id, result.tokens_used)
    return result
```

### 🕒 RATE LIMIT (PER MINUTE)

🛡️ Protects: your servers

# Status: 429

💬 "Slow down, retry soon"

🔄 Resets: seconds

*Temporary, retryable*

### 📅 QUOTA (PER BILLING PERIOD)

🛡️ Protects: your margins

# Status: 402

💬 "Limit reached, upgrade"

🔄 Resets: billing cycle

*A business boundary*



AI Coach John

Follow



PROITBRIDGE  
DRIVE FOR BETTER LEADERS

🔄 Repost

# Middleware: Logging & Timing

## ▼ CROSS-CUTTING CONCERNS IN ONE PLACE

```
@app.middleware("http")
async def observability_middleware(request: Request, call_next):
    request_id = str(uuid.uuid4())[:8]
    request.state.request_id = request_id

    start = time.perf_counter()
    response = await call_next(request)
    duration_ms = round((time.perf_counter() - start) * 1000)

    log.info("request_finished",
            status=response.status_code, duration_ms=duration_ms)

    response.headers["X-Request-ID"] = request_id
    response.headers["X-Response-Time-Ms"] = str(duration_ms)
    return response
```

## 📦 MIDDLEWARE WRAPS EVERY REQUEST

↓ incoming

assign request\_id · start timer

CORS → auth → rate limit → endpoint

log status · attach headers · stop timer

↓

**X-Request-ID: a3f91e2b**

**Give this to users so support tickets map straight to a trace.**



AI Coach John

Follow



PROITBRIDGE  
DRIVEN BY BETTER TOOLS

🔄 Repost

# Health Checks That Mean Something

## ❗ "THE PROCESS IS ALIVE" IS NOT THE SAME AS "THE SERVICE WORKS"

```
@app.get("/health/live")
async def liveness():
    # Is the process running? Used to decide whether to restart it.
    return {"status": "alive"}

@app.get("/health/ready")
async def readiness():
    # Can it actually serve traffic? Used to decide routing.
    checks = {}
    checks["models_loaded"] = "embedder" in ml_models
    try:
        await asyncio.wait_for(vector_store.aping(), timeout=2.0)
        checks["vector_store"] = True
    except Exception:
        checks["vector_store"] = False

    healthy = all(checks.values())
    return JSONResponse(
        status_code=200 if healthy else 503,
        content={"status": "ready" if healthy else "degraded"},
    )
```

### 🔌 /HEALTH/LIVE

If fails:

**Orchestrator RESTARTS the container.**

### 🚫 /HEALTH/READY

If fails:

**Orchestrator STOPS SENDING TRAFFIC (does not restart).**

⚠️ **Getting these backwards causes restart loops: if /live checks the vector DB and the DB blips, every container restarts for a problem restarting can't fix.**



AI Coach John

Follow



PROITBRIDGE  
DRIVEN BY BETTER LEARN

↻ Repost

# Configuration & Secrets

## ✓ TYPED SETTINGS, LOADED ONCE, VALIDATED AT STARTUP

```
class Settings(BaseSettings):
    # secrets: from environment, never hardcoded
    llm_api_key: str
    vector_db_url: str

    # tunables: sensible defaults
    model_name: str = "claude-sonnet-4-6"
    max_tokens: int = 1024
    environment: Literal["dev", "prod"] = "dev"

    class Config:
        env_file = ".env"

@lru_cache
def get_settings() → Settings:
    return Settings() # validated once
```

## ✗ FAIL MID-REQUEST

```
os.getenv("LLM_API_KEY")
```

Returns None. App boots fine. First user request crashes.

## ✓ FAIL AT STARTUP

```
Settings(llm_api_key: str)
```

Missing key → app refuses to start with clear error. You find out in CI, not from a customer.



# The Concurrency Model

## ⚠️ WORKERS, THE EVENT LOOP, AND WHERE AI SERVICES GO WRONG

`uvicorn main:app --workers 4`

### WORKER 1

Event loop  
~100s reqs

Own model  
2 GB

### WORKER 2

Event loop  
~100s reqs

Own model  
2 GB

### WORKER 3

Event loop  
~100s reqs

Own model  
2 GB

### WORKER 4

Event loop  
~100s reqs

Own model  
2 GB

⚠️ 8 GB total for a 2 GB model

☁️ API-backed LLM (nothing big in RAM) → more workers fine

```
uvicorn main:app --workers 4
```

🏠 Local model in memory → rely on async concurrency

```
uvicorn main:app --workers 1
```



# Testing Your API

## ⚡ FAST TESTS WITH THE PIPELINE MOCKED OUT

```
@pytest.fixture
def client():
    app.dependency_overrides[get_pipeline] = lambda: FakePipeline()
    yield TestClient(app)
    app.dependency_overrides.clear()

def test_rejects_blank_question(client):
    r = client.post("/query", json={"question": " "})
    assert r.status_code == 422

def test_returns_expected_shape(client):
    r = client.post("/query", json={"question": "what is RAG?"})
    body = r.json()
    assert set(body) >= {"answer", "citations", "tokens_used"}
```

## ✔ WHAT TO TEST AT THE API LAYER

- ✔ Validation rejects bad input (422)
- ✔ Response shape matches the model
- ✔ Auth blocks missing/invalid keys (401)
- ✔ Rate limits fire (429)
- ✔ Errors return right status, not 200
- ✔ Headers attached (request ID, timing)
- The actual answer text (eval suite's job)



# Deployment Dockerfile

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app/ ./app/

# start-period must cover model loading, or restart loop
HEALTHCHECK --interval=30s --timeout=5s --start-period=90s \
  CMD curl -f http://localhost:8000/health/ready || exit 1

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000", \
  "--workers", "2", "--timeout-keep-alive", "65"]
```

## ! THREE SETTINGS PEOPLE GET WRONG

**--host 0.0.0.0**

Without it, unreachable outside the container.

**--start-period 90s**

Model loading takes 60s → container killed at 30s → **infinite restart loop**

**--timeout-keep-alive**

Streaming connections stay open; too low a value cuts long responses mid-stream.





# Complete Production Structure

## EVERYTHING ASSEMBLED

```
rag-api/
├── app/
│   ├── main.py           # app, lifespan, middleware
│   ├── config.py         # pydantic Settings
│   ├── dependencies.py   # get_pipeline, rate limiter
│   ├── models/
│   │   ├── requests.py
│   │   └── responses.py
│   ├── routers/
│   │   ├── query.py
│   │   └── health.py
│   └── services/
│       ├── pipeline.py   # the RAG logic itself
│       └── llm_client.py # retries + timeouts
├── tests/
├── Dockerfile
└── pyproject.toml
```

## THE REQUEST PATH

client

- ↓ middleware request ID, structured logs
- ↓ auth API key → user
- ↓ rate limit per-key + per-IP
- ↓ validation Pydantic → 422 on failure
- ↓ endpoint **parse, call service, shape response**
- ↓ service RAG pipeline
- ↓ response

**Keep endpoints thin. Validate, call one service, shape response. Business logic in routes makes APIs untestable.**



AI Coach John

Follow



PROITBRIDGE  
DRIVER FOR BETTER LEADERS

Repost

# Common Mistakes

- ⚠ Loading model in endpoint instead of lifespan
- ⚠ async def endpoint with blocking call
- ⚠ Returning 200 with error message in body
- ⚠ No timeout on LLM call (hangs forever)
- ⚠ Health check hits LLM (their outage restarts you)
- ⚠ Leaking stack traces to clients

## 🔍 SYMPTOM → USUALLY MEANS

| SYMPTOM                    | CAUSE                               |
|----------------------------|-------------------------------------|
| First request slow         | Model loading per request           |
| Restart loops              | Health check start-period too short |
| Streaming works local only | Missing X-Accel-Buffering: no       |
| OOM with 4 workers         | Each worker holds own model copy    |
| Requests still queue       | Blocking call inside async def      |
| Can't debug report         | No request ID returned to client    |



# Production Readiness Checklist

## SETUP & VALIDATION

- ☐ Models loaded once in lifespan
- ☐ Pydantic validation on request/response
- ☐ Settings typed & validated at boot
- ☐ API docs reviewed for accuracy
- ☐ Worker count matched to RAM

## REQUESTS

- ☐ Streaming proxy headers correct
- ☐ Timeouts on every external call
- ☐ Semaphore capping LLM calls

## RESILIENCE

- ☐ Rate limits (429) / quotas (402)
- ☐ Custom exception handlers (no traces)
- ☐ /live and /ready separated
- ☐ Docker start-period covers model load

## OBSERVABILITY

- ☐ Request ID in logs and header
- ☐ Tokens, cost, latency logged

